PROJECT BASED LEARNING REPORT

On

Virtual Memory Management Simulator

Submitted by:

1. **Amit Kumar (1/24/SET/BCS/179)**
2. **Lucky kumar (1/24/SET/BCS/193)**
3. **Ankul Raj (1/24/SET/BCS/178)**

Subject: Operating System [5.0CE202C01H]

Academic Year: 2025–2026, Even Semester

Under the Guidance of:
Faculty Name: **Dr. Vandana (**Assistant Professor**)**
Department of Computer Science and Engineering
Manav Rachna International Institute of Research and Studies

ABSTRACT

This project implements a Virtual Memory Management Simulator that demonstrates different page replacement algorithms such as FIFO, LRU, and Optimal. The system simulates how an operating system manages limited RAM and decides which memory page to replace when new pages are required.

The project uses arrays, loops, and logical decision-making to simulate memory frames and track page faults. It highlights how efficient memory management improves system performance and reduces page faults. This project demonstrates the practical application of Operating System concepts in solving real-world problems related to memory allocation and optimization.

INTRODUCTION

Virtual memory is an important concept in operating systems that allows programs to use more memory than physically available in RAM. When the memory is full and a new page needs to be loaded, the system must decide which existing page to remove.
This decision is made using page replacement algorithms. In this project, we simulate three commonly used algorithms:

- FIFO (First In First Out)
- LRU (Least Recently Used)
- Optimal Page Replacement

The simulator helps understand how different algorithms behave and how they impact performance in terms of page faults.

OBJECTIVES

- To simulate virtual memory management using programming
- To implement page replacement algorithms (FIFO, LRU, Optimal)
- To understand the concept of page faults and memory frames
- To compare efficiency of different algorithms
- To demonstrate real-world working of OS memory management

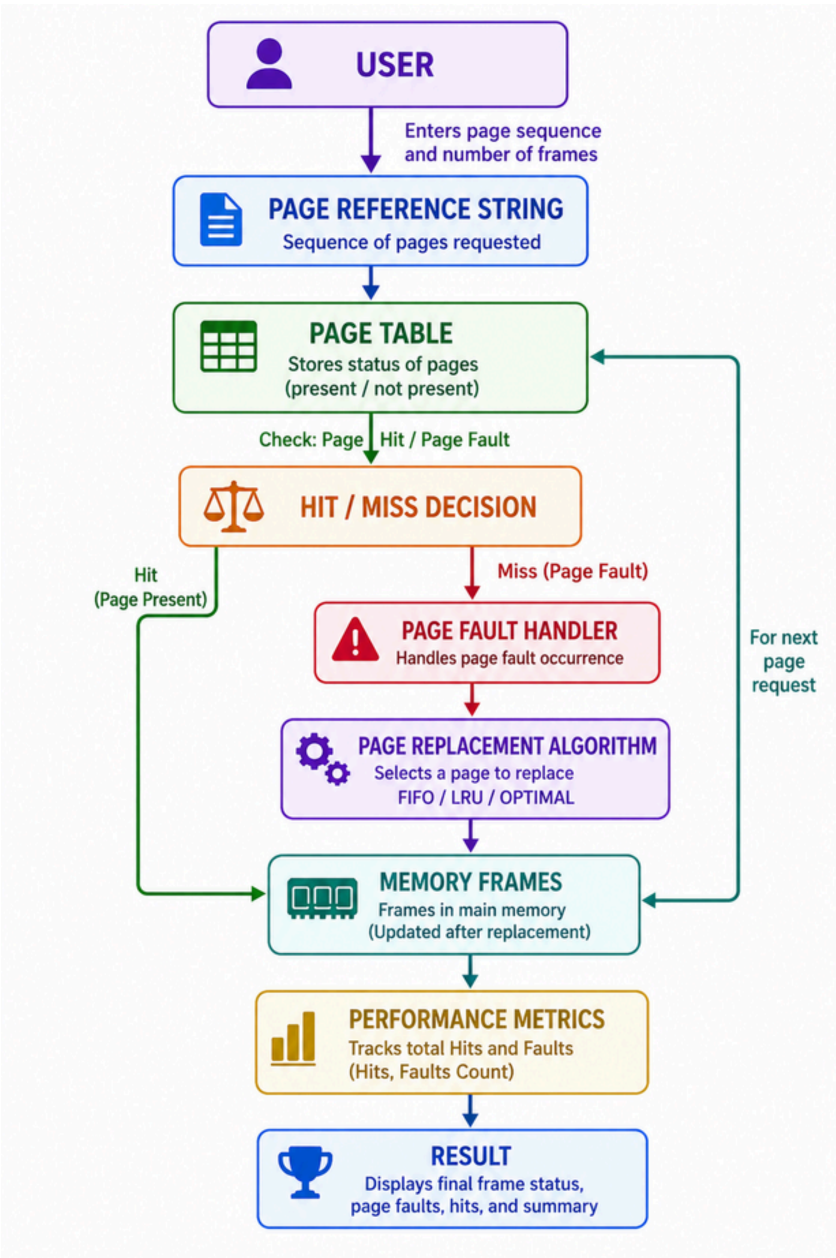
SYSTEM ANALYSIS

In real systems, RAM is limited, and programs require efficient memory allocation. Without proper algorithms, memory usage becomes inefficient and increases page faults.

This system simulates memory using an array representing frames. When a page is requested:
• If it is already in memory → No fault
• If not → Page fault occurs

Algorithms decide which page to remove:
• FIFO removes oldest page
• LRU removes least recently used page
• Optimal removes page not used for longest future time
This automated simulation provides accurate and fast decision-making compared to manual calculation

ER DIAGRAM AND SCHEMA DESIGN



Entities
- Pages
- Memory Frames
- Page Table
- Page Replacement Algorithm

Relationships
- Pages are loaded into frames
- Frames update memory state
- Algorithm decides page replacement

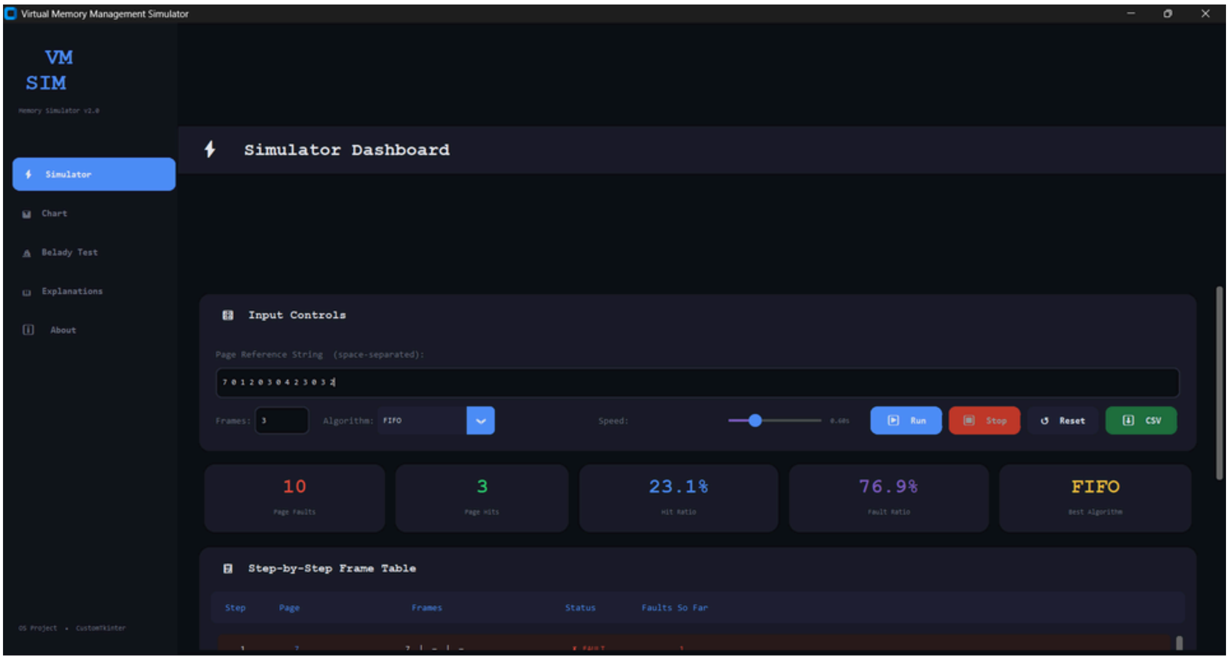
Conceptual Flow
Page Request → Check Memory → Page Fault → Apply Algorithm → Replace Page → Update Memory

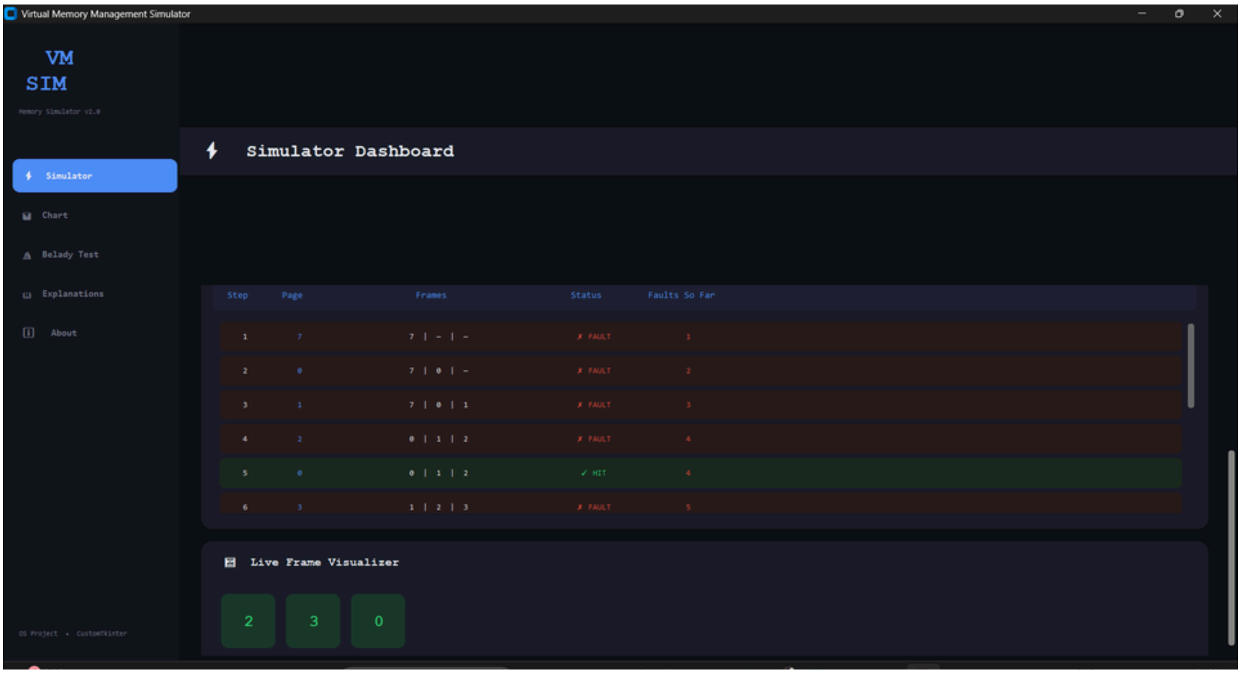
IMPLEMENTATION

- Memory frames are represented using arrays
- Page sequence is taken as input from user
- Separate functions implemented for:
  - FIFO algorithm
  - LRU algorithm
  - Optimal algorithm
- Loop traversal is used to check page presence
- Counters are used to calculate page faults
- Output displays:
  - Frame status after each step
  - Final result (page faults and hits)

OUTPUT

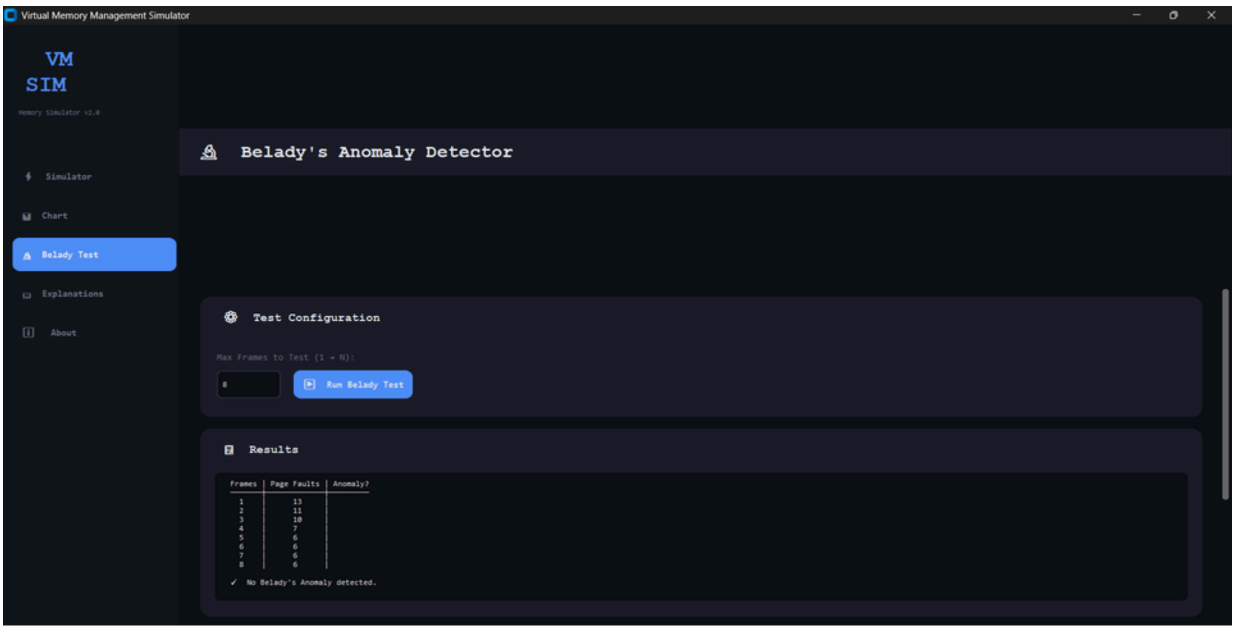
Stimulator Dashboard

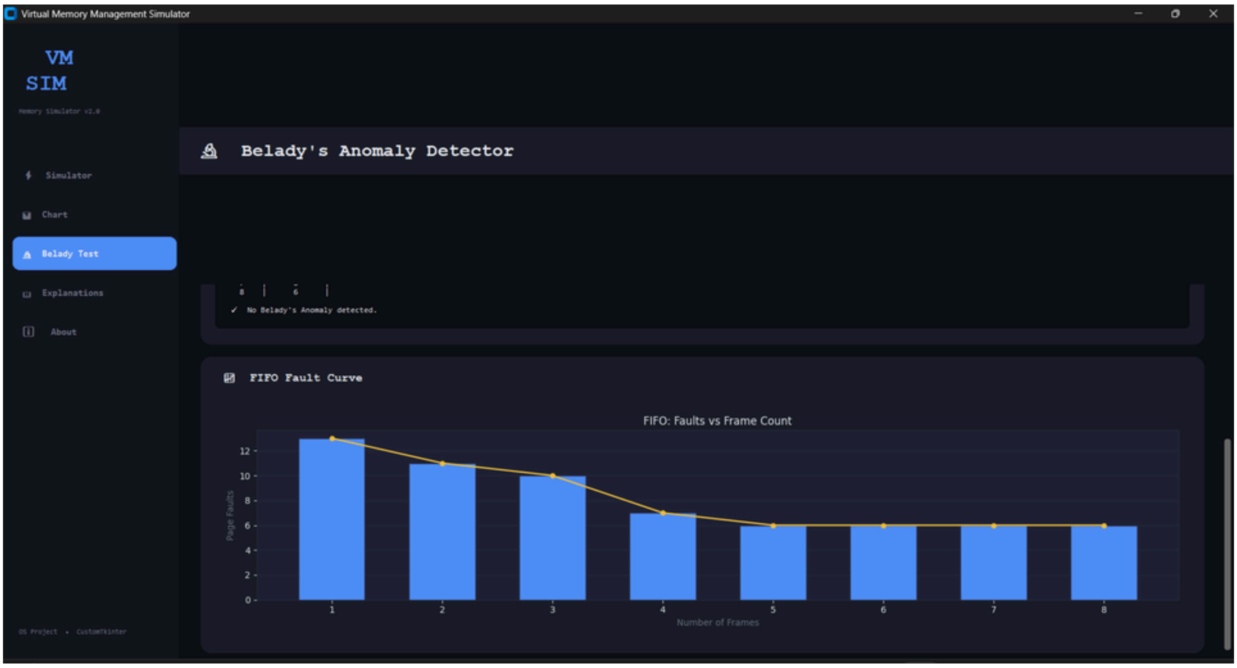




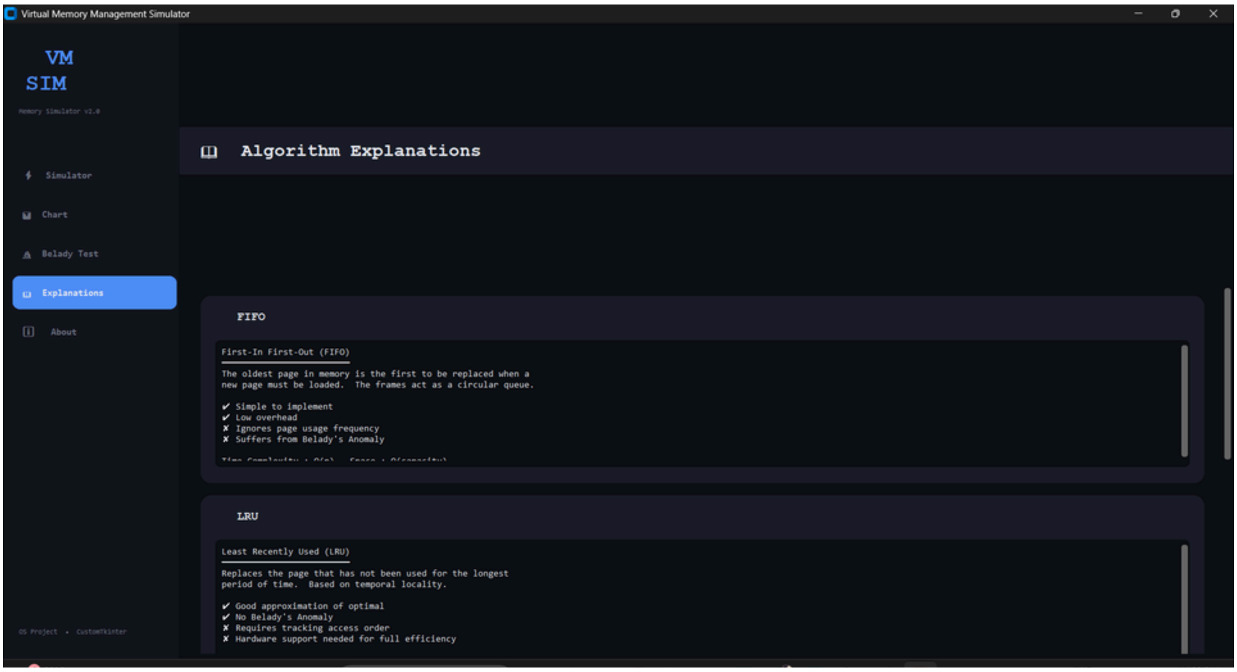
Chart

Belady Test





Explainations



About

RESULTS AND DISCUSSION

The simulator successfully demonstrates how different page replacement algorithms work.

- FIFO is simple but may lead to more page faults
- LRU performs better by using past usage data
- Optimal gives best performance but is theoretical

The comparison shows how algorithm choice affects system efficiency.

CONCLUSION AND FUTURE WORK

This project demonstrates the importance of memory management in operating systems. The simulation clearly shows how page replacement algorithms help in efficient use of RAM.

Future Scope

- Add graphical user interface (GUI)
- Real-time visualization of memory frames
- Implement additional algorithms (Clock, LFU)
- Extend to multi-process memory simulation
- Integrate with real OS-level data

REFERENCES

- Operating System Concepts
- GeeksforGeeks – Page Replacement Algorithms
- Tutorialspoint – Operating System Tutorials
- W3Schools – Basic Programming Concepts
- Classroom notes and lecture material
- Modern Operating Systems
- NPTEL – Operating System Lectures
- MIT OpenCourseWare – Operating Systems Course